

Pocketful Delivery Line

One spec in. A tested, deployed and watched wallet service out, built by eight Band seats on one Docker host.

coordinator

architect

tester

developer

reviewer

qa-explorer

release-manager

sre-monitor

Build loop · Ship and watch loop

THE TASK

A wallet where money is never created, lost or spent twice

1

JSON API

Idempotent writes on 5 paths, exact split rounding, atomic settlements, export/import

2

Browser UI + holds

Recovery from lost responses and stale reads; authorizations with partial captures

3

Statements + corrections

Effective vs recorded time, historical overdraft checks, frozen snapshots

4

Refunds + batch corrections

Whole-settlement corrections with strict error precedence

HOW IT IS JUDGED

50%

Factory

generic mandates, how far it got, reusable FACTORY.md

25%

App

presentation-ready UI and maintainable code

25%

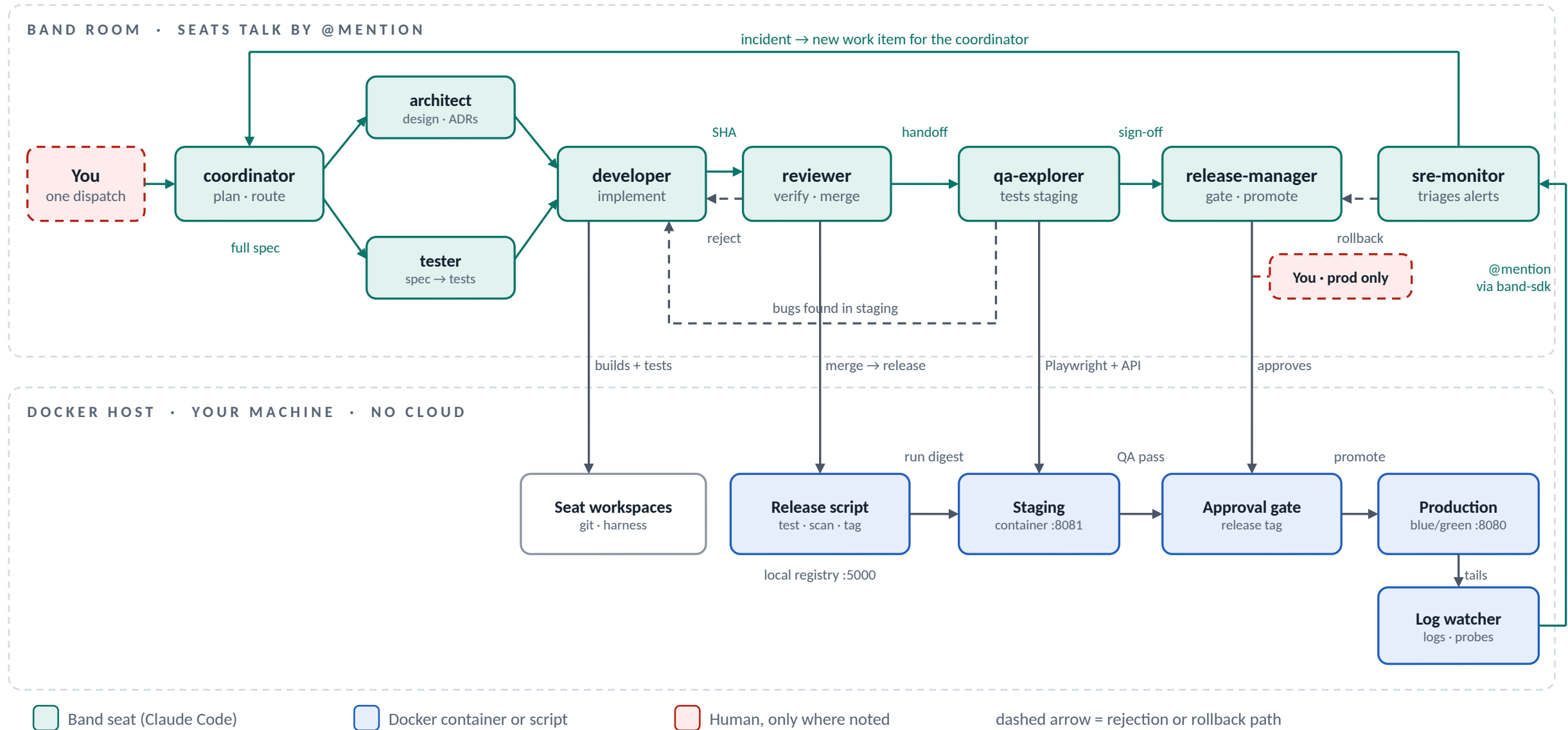
Agent teamwork

shared work, real review, no human steering

Submissions close **Mon Oct 5, 23:59 PDT**

END-TO-END FLOW

Eight seats in one Band room, one Docker host underneath



THE SEATS

Each seat owns one job and says when it rejects work

Seat	Model	Owns	Rejects when
coordinator	c laude - opus - 5 - 5	Plans the stage, routes handoffs, writes the outcome	A handoff is missing the spec, SHA or evidence
architect	c laude - opus - 5 - 5	Design delta, ADRs, invariants	The design breaks a stated invariant
tester	c laude - sonnet - 5	Black-box tests from the spec	A spec rule has no test
developer	c laude - sonnet - 5	One work item at a time, commits	—
reviewer	c laude - opus - 5 - 5	Clean checkout, harness, suite, merge	Any check fails or an invariant breaks
qa-explorer	c laude - sonnet - 5	Tests the staging container by hand	A UI state is wrong, a retry double-spends, totals drift
release-manager	c laude - sonnet - 5	Only seat that runs or switches containers	The signed-off digest ≠ the promoted digest
sre-monitor	c laude - sonnet - 5	Triages alerts from the log watcher	A threshold is breached → rollback or incident

Harness for every seat: Claude Code in Band Desktop. Mandates name no endpoints, fields or error codes, so the harness vocabulary scan passes.

Same seats, same mandates: one node decides the mode

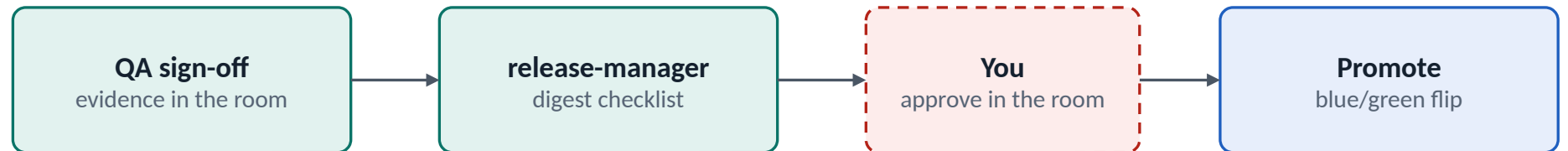
JUDGED RUN

dark factory, no human wait



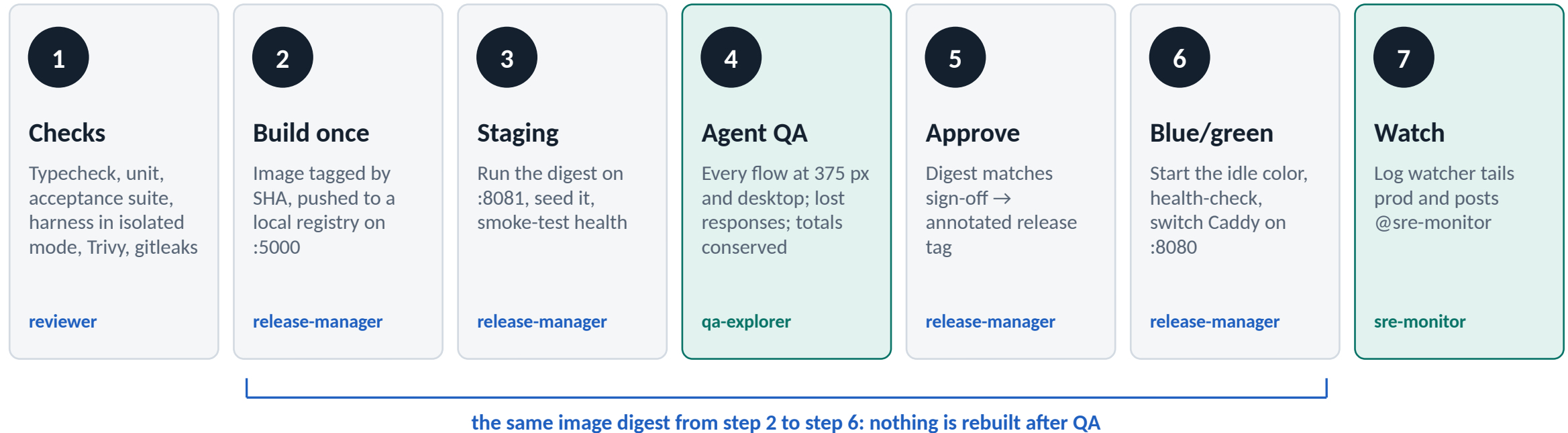
PRODUCTION MODE

human gate on



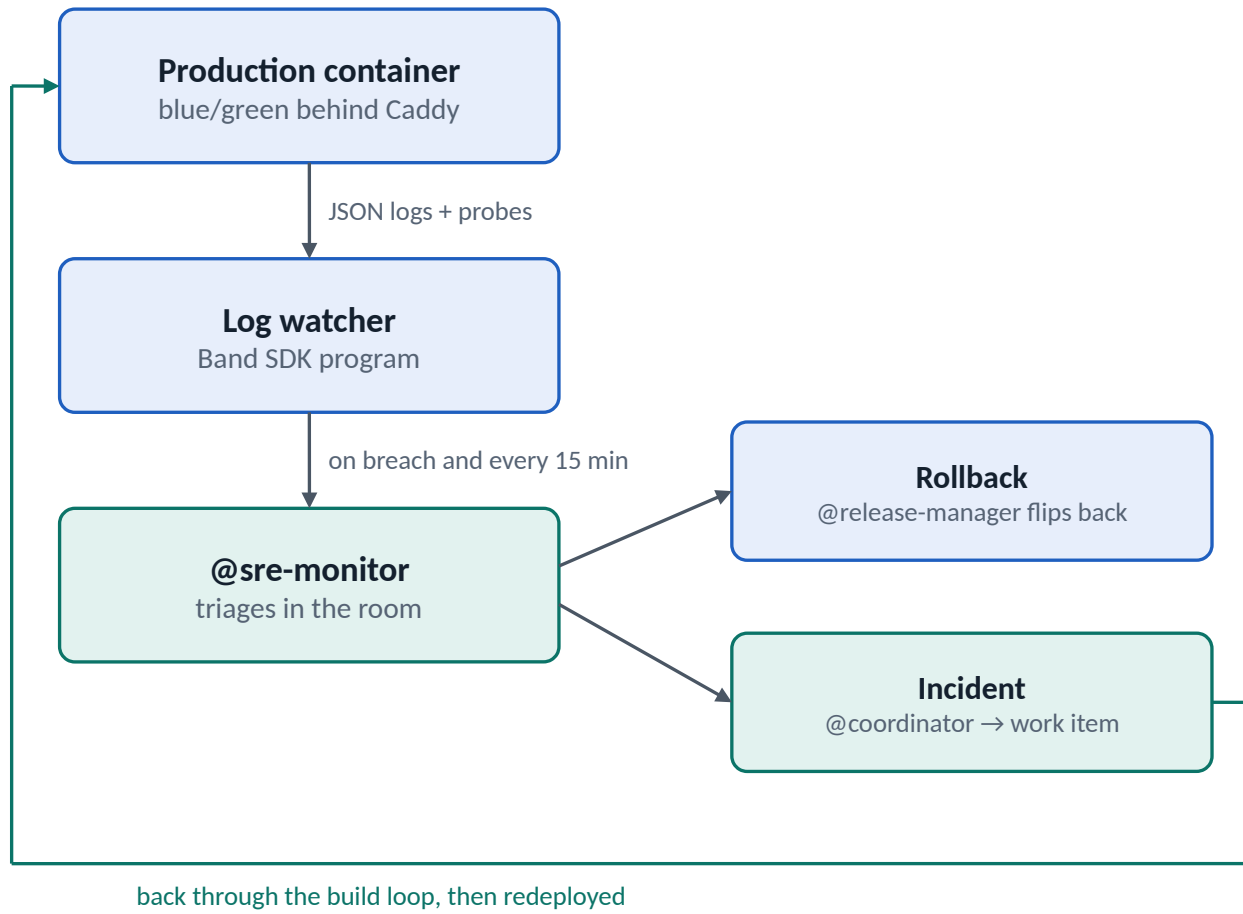
Why two modes. The judged run allows one human input per stage, the dispatch. An "approve" click would count as steering. In both modes release-manager checks that the digest QA signed off on is the digest being promoted and records it in a release tag. In production mode the release script also waits for your approval. The seat never asks you.

Seven steps from merge to watched production



Staging and production run after the reviewer accepts a stage, so they add evidence without blocking the stage report.

Production logs become @mentions in the room



MONEY-SPECIFIC SIGNALS

- **Conservation probe**
sum of wallet balances equals the seeded total
- **Replay mismatch**
an idempotent replay returned a different body
- **Any 5xx**
the spec forbids them, even under load
- **Refusal spike**
sudden rise in insufficient-funds rejections
- **Slow requests**
p99 approaching the 5 s request timeout

One container, designed for stage 4 from stage 1



In-memory state, single-threaded writes

Each write runs synchronously from claiming the idempotency key to commit, so concurrent requests serialize with no locks. Only password hashing is async, outside the critical section.



Append-only payment ledger

Payments keep a list of revisions with effective and recorded times. One function computes any balance as of a time and as known at a time, which stages 3 and 4 reuse.



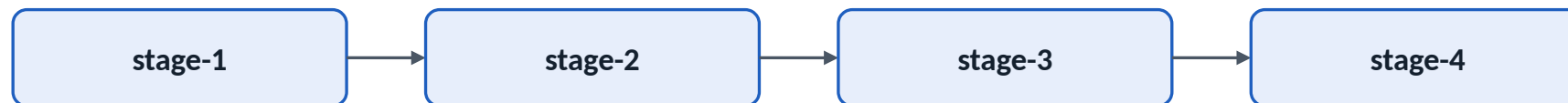
Idempotency before validation

Parse, authenticate, then resolve the key. A replay returns the stored response, a different body is refused, a failed request frees the key.



UI in the same process

Server-rendered pages with bundled fonts and CSS. The pay form keeps its key until a field changes, so a lost response can be retried safely.

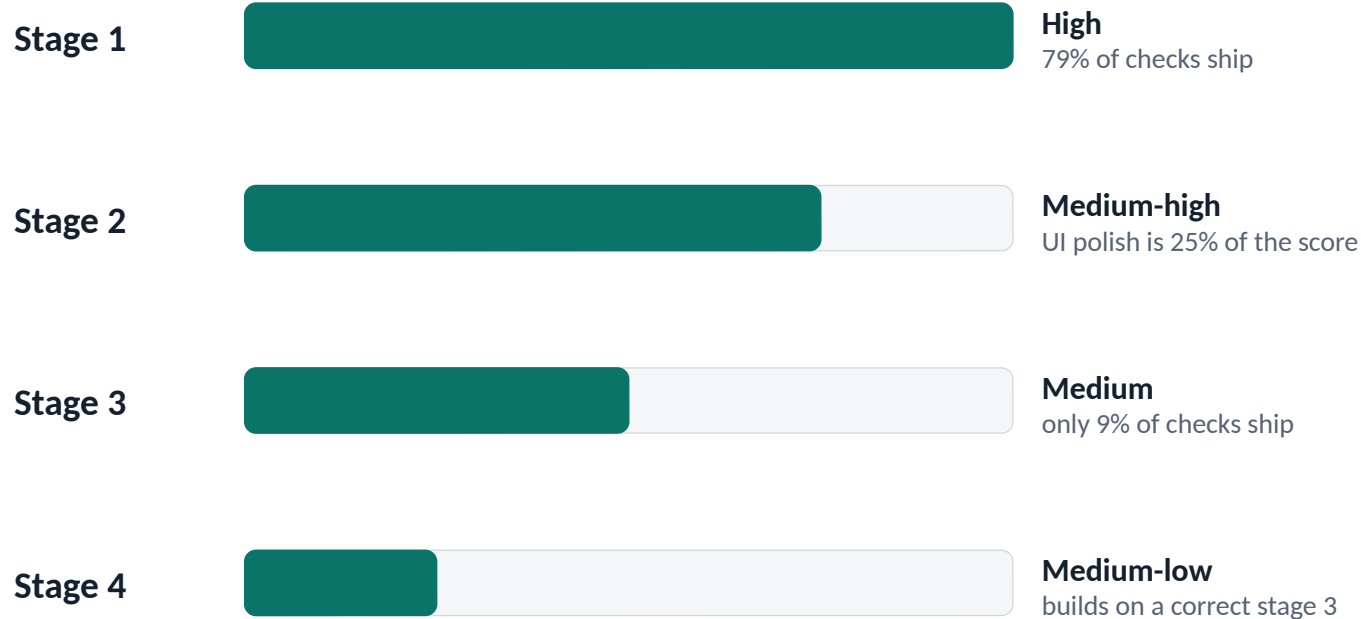


Each folder is the previous one copied forward and extended. It passes every earlier suite and never the next one.

FEASIBILITY

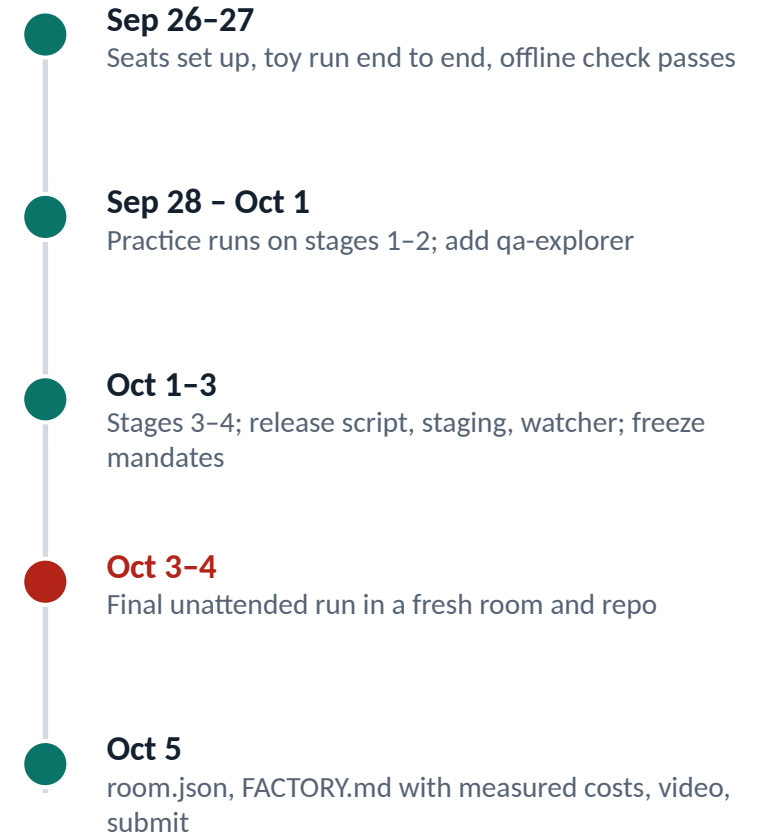
Stages 1 and 2 are likely; autonomy is the real risk

CONFIDENCE BY STAGE (JUDGEMENT, NOT MEASURED)



The deciding risk is autonomy: one seat stalled on a permission prompt or asking a question spoils the judged run. Rehearse on the toy first.

PLAN TO THE DEADLINE



What can disqualify the entry, and how the factory handles it



One human input per stage

No approvals or hints after the dispatch. The coordinator decides and logs ambiguities.



Code must come out of the room

Every commit traces to a handoff. Nobody edits stage folders by hand.



Build to the spec, not the tests

The tester writes tests from the spec; shipped checks are only a signal.



Mandates must be generic

No endpoints, fields or error codes in mandates; run the offline check after every edit.



Docker Sandboxes isolate Docker

Seats that touch containers run on host Docker so staging is visible to all.



room.json is public

No credentials in messages; read the file before committing it.

NEXT

From spec to watched production, without a human in the loop

Write the eight generic mandates
and pass the offline vocabulary check

Build the release script
local registry, staging, Caddy blue/green, log
watcher

Rehearse on the toy track
then the final unattended Pocketful run